

Raytraced Voxel Radiosity

Ivo te Sligte
s4070763

04. December 2025

Abstract

Global Illumination has long stumped graphics programmers due to its incredibly high cost. In a typical path traced implementation millions of rays are shot into the scene every frame. This project instead uses an approach inspired by radiosity. A world-space cache is used on top of a voxel world for fast ray traversal and extensive ray re-use.

Running

The source code for this project is available at <https://github.com/IvotSligte/cg-fp>

This program has been tested on the PC with service tag HW7LR34.

It should work on all Linux PCs running X11, OpenGL 4.5, and CMake 3.14.

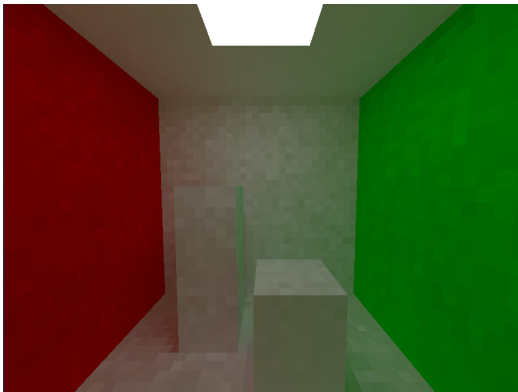


Figure 1: Cornell box.

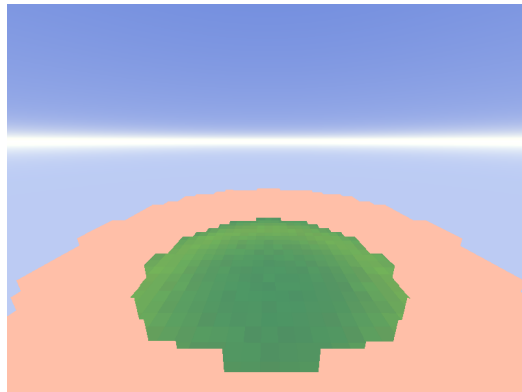


Figure 2: Emissive donut with visible skybox.

Summary

This project consists of a compute shader that calculates per-voxel-face lighting using raytracing and a fragment shader that renders the voxel world from the perspective of the camera by casting rays into the scene. This approach creates plausible results without too much performance overhead. However, there are still some limitations with regards to screen resolution and energy conservation.

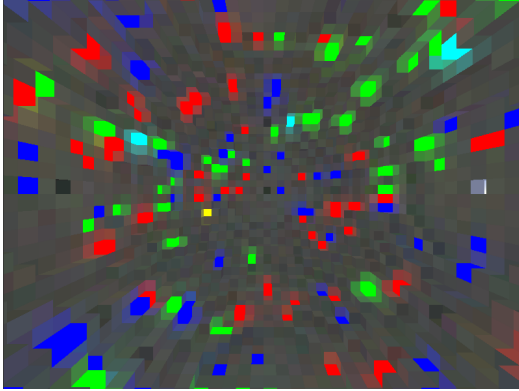


Figure 3: Inverted sphere scene with many light sources.



Figure 4: Simple scene with spherical light source.

Implementation

Libraries

This project depends on a couple libraries to make development easier. SDL2[1] and OpenGL[2] are used as cross-platform APIs for window handling and graphics visualisation respectively. As OpenGL is not quite accessible without a wrapper that handles includes, GLAD[3] is used for this purpose. Lastly, GLM[4] is used for CPU-side vector math.

Note that this project was not made in collaboration with anyone else.

Boilerplate

Computer graphics projects usually have a lot of boilerplate to make the shaders work. This project is no different. The boilerplate is written in C++ and consists of files for interacting with SDL and OpenGL while preserving programmer sanity as well as possible. The camera- and scene-related logic is also in separate files.

Scenes

Four scenes are included in the project and can be switched between using the **N** key. They are shown in Figure 1, Figure 2, Figure 3 and Figure 4. More can be added by extending `scene.h`.

Shaders

The vast majority of the project complexity lies in shaders. Each scene consists of a fixed-size voxel grid (32^3) and each voxel face has its own albedo, emission and colour (irradiance) values. In the compute shader that handles world-space lighting updates, each voxel face shoots out a fixed number of rays every frame using simple cosine-weighted Monte Carlo sampling. These rays either return a colour value depending on whether they missed everything (sky colour) or hit another voxel. The returned values are accumulated over many frames so that eventually the entire scene is properly lit and no longer noisy.

To render to the scene a full-screen quad vertex shader and fragment shader are used. The fragment shader casts a ray per pixel and retrieves the colour value of the voxel it hit, or the colour of the sky according to a mathematical function.

Interactivity

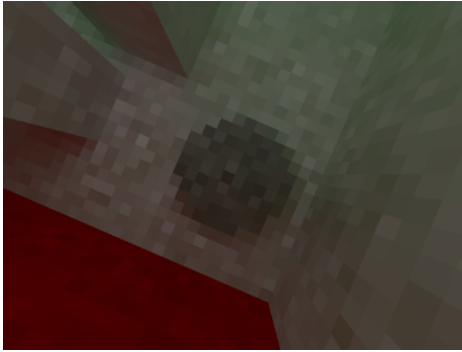


Figure 5: Darkening from the camera's shadow in the Cornell box scene.

The user can move the camera around with the typical **WASD** keys and mouse. The camera interacts with the scene by casting a small shadow on surfaces in the scene. Due to the temporal accumulation of the world-space cache, it can take a second for the shadow to become visible even when the camera is immobile.

Limitations

For simplicity, at the moment only one face direction is updated per frame. Since face colours are zero-initialized, this means slow convergence and a dark scene for the first hundred or so frames on scene enter. This can be fixed easily by casting rays for every face, multiplying the number of rays cast by 6. The performance of the world-space lighting update shader is more than good enough to allow this. The only reason why this has not been done is because this requires expanding the random ray uniform buffer. Doing so massively slows down compilation time on higher end GPUs. Hypothetically, this is because their drivers try to optimise the large arrays that are accessed in a trivially predictable pattern. Using more than 256 random rays has crashed the RTX 2060 used for testing during shader compilation. A laptop without a GPU that was also used for testing had constant compilation time regardless of random ray count.

The greatest obstacle to runtime performance is the fragment shader that casts one ray per pixel. Due to the high resolution of modern screens, it casts over two million rays per frame on a typical 1920x1080 screen. For comparison, the world-space cache casts only half a million rays per frame. The fragment shader is not required to cast rays, however, and can be replaced by a hardware mesh rendering pipeline that renders the voxel scene as a mesh.

Another performance issue is the fact that the light update shader is invoked for every face in the volume, rather than only on the surface. A check is done in the shader, so no rays are sent for air or occluded faces, but there are still many more inactive threads than necessary due to the workgroup nature of compute shaders.

Lastly, energy conservation is not taken into account. The result is that scenes gradually increase in brightness over time. This is particularly visible in Figure 4.

References

- [1] “SDL2 – Simple DirectMedia Layer 2.0.” 2025.
- [2] Khronos Group, “OpenGL – The Industry Standard for High-Performance Graphics.” 2025.
- [3] “GLAD – Multi-Language GL/GLES/EGL/GLX/WGL Loader-Generator.” 2025.
- [4] G-Truc, “GLM – OpenGL Mathematics.” 2025.